

Investigating Contrastive Learning for Fish Identification and Classification

Joseph Goldblatt*
Department of Computer Science
University of Cape Town
South Africa

Patrick Marais†
Department of Computer Science
University of Cape Town
South Africa

Jonathan Shock†
Department of Mathematics and
Applied Mathematics
University of Cape Town
South Africa

ABSTRACT

This paper investigates the application of contrastive learning, a self-supervised machine learning technique, to the task of classifying fish species from underwater video. Specifically, we investigate classification of fish species from a custom dataset of underwater videos collected off the East Coast of South Africa by the South African Institute for Aquatic Biodiversity (SAIAB). This dataset contains hundreds of hours of unlabelled underwater footage, with sparse species annotations. The dataset is well suited to testing the application of contrastive learning, which has been shown to produce high classification accuracy when trained on a large unlabelled dataset and a small labelled one. We therefore implemented the SimCLR contrastive learning framework, first proposed by Google in 2020. We trained a ResNet-50 model using a large unlabelled dataset extracted from the SAIAB footage, supplemented with supervised training on a small annotated dataset. The model achieved a mean classification accuracy (across ten random seeds) of 70.65% on the labelled dataset of 38 classes. However, this accuracy was matched by a state of the art YOLOv11-n classifier model, suggesting that although there is scope for the application of self-supervised learning to this task, out-performing current supervised classifiers may require implementing a more modern self-supervised framework.

1 INTRODUCTION

Fishery managers, marine biologists, ecologists and conservationists all require accurate estimates of fish population size and composition [24]. Recently, there has been a growing interest in the use of Remote Underwater Video (RUV) methods for fish population monitoring [6]. Specifically, stereo Baited Remote Underwater Video (stereo-BRUV) systems have been shown to be effective for fish population surveys [7]. Stereo-BRUV systems are non-extractive and allow experts to analyse the collected data off-site and asynchronously [7]. However, a consequence of the uptake of RUV monitoring techniques is a rapid increase in the amount of data to be analysed by experts [30]. Therefore, a method to automate the identification, classification and counting of fish from underwater video would greatly increase the efficacy of RUV monitoring techniques [30]. Recently, a range of Deep Learning (DL) based computer vision models have been proposed to address this issue [30].

The South African Institute for Aquatic Biodiversity (SAIAB) [33] is a nationally funded marine research unit, specialising in marine ecology, conservation and biodiversity. Under their Marine Remote Imagery Platform (MaRIP) the SAIAB has collected approximately

8,000 hours of remote underwater footage from multiple locations on the South African continental shelf¹. Many of MaRIP's data collection sites are located in South Africa's Marine Protected Areas (MPA) [22], and therefore analysis of this data supports research into, among other things, climate change and fishing regulation.

Currently, analysis involves an expert (called a tape-reader) watching entire videos and manually identifying, classifying and annotating frames. The SAIAB analysis of a video primarily consists of identifying, for all observed aquatic species, the maximum number of members of this species observed at once (this is called the MaxN of the particular species) [6]. Additional data is collected along with this, such as the length of all species' members in the MaxN frame and, occasionally, the activity of species' members during the MaxN frame¹.

Due to the resource-intensiveness of analysing such large quantities of data, only approximately half of MaRIP's videos have been analysed¹, which is a severe road-block to further work. This project focuses specifically on investigating possible DL solutions to the automation of this analysis, in the hope of streamlining the analysis process.

The primary goal of this project is to investigate the feasibility of producing DL tools for identifying and classifying fish from the SAIAB dataset. The hope being that these foundational tools can then be used to perform more complex tasks such as fish counting and measurement. However, the SAIAB's dataset precludes us from using traditional DL approaches to solve this problem. Typical DL approaches require large amounts (in the order of hundreds of thousands or millions) of labelled examples to learn from. Although the available dataset is large (approximately 16TB of high-quality underwater footage), it is very sparsely annotated, with only ~ 2,500 partially labelled frames. Additionally, the annotations of underwater samples are simply points with species labels, rather than the bounding boxes traditionally required for vision-based DL models.

To solve this dataset issue, we investigate Self-Supervised Learning (SSL), a Machine Learning (ML) paradigm wherein unlabelled data is first used to train a DL model to extract useful features from examples, after which the model is fine tuned on a small set of labelled data to effectively utilize these features. This paradigm allows for unlabelled data in the dataset to be leveraged for learning. Specifically, we investigated the SimCLR contrastive learning framework introduced by Google in 2020 [8].

*Student.

†Supervisor.

¹E. Heyns-Veale, private communication Aug. 18, 2025

2 RELATED WORK

2.1 Background

Remote Underwater Video for Fish Surveying. Traditionally, fish population surveys were performed using Underwater Visual Census (UVC) by divers in shallow water; and extractive methods such as trawling in deeper water [6]. However, both of these approaches pose issues. UVC methods firstly require the presence of highly trained research-specialists; and secondly are susceptible to the diver’s subjective observation biases [7]. Extractive methods are not suitable for complex, or ecologically vulnerable regions due to the damage that can be caused to fish populations and habitats [7]. Cappelletti et al [6] discuss the recent popularity of RUV methods for fish population surveys. RUV methods are non-extractive, relatively inexpensive and allow for asynchronous identification of fish by off-site experts. RUV methods involve the lowering of a rig containing one or more underwater cameras to an area of interest [6]. BRUV methods include a bait attached to the rig - often on an extended arm - to attract aquatic life to the area being recorded [7]. Stereo-BRUV rigs include two cameras positioned at an angle to one another to provide depth information. Langlois et al [19] have shown that fish size and position can be accurately measured from footage produced by stereo-BRUV rigs.

Underwater Object Detection. Underwater object detection tasks are made difficult by a range of factors specific to the setting of the input images [35]. For example, Xu et al [35] mention blur, low contrast and hazing in images as factors which, among others, make underwater object detection challenging. Additionally, both Jian et al [16] and Fu et al [10] discuss the deficiencies of current underwater image and video datasets. Both papers discuss the lack of large scale, high quality, annotated datasets [10, 16]. This confirms that DL based approaches to underwater object detection are particularly difficult due to the fact that typically DL models require extensive labelled datasets to accurately “learn” specific feature sets [20].

2.2 Deep Learning for Fish Identification and Classification

In the last decade, there have been numerous studies published which investigate the application of DL to fish detection and classification in underwater images and videos [30]. There are two broad categories of DL models which have been applied to the problem of fish identification: YOLO (You Only Look Once) based models and R-CNN (Region-based Convolutional Neural Network) based models [30].

YOLO models, unlike object detection models such as R-CNN, perform both object detection and classification through a **single** convolutional neural network [28]. Because of this single pass architecture, YOLO models are more time efficient than two-stage models such as R-CNN based models. This makes YOLO an attractive option for real-time or low resource object detection. Additionally, YOLO models are open-source, easy to implement and rank highly on benchmarks for accuracy [25]. It is likely for these reasons that many researchers have chosen to build fish identification models that integrate YOLO models. Zhang et al [38], Marrable et al [23],

Zheng et al [40], and Qin et al [27] have all produced YOLO based fish identification models.

Notable papers in the literature include Jalal et al [15], where the researchers achieved significant success in fish identification and classification using YOLOv3 in combination with a Gaussian Mixture Model (GMM) and Optical Flow algorithm; Knausgard et al [18], where the researchers proposed a system which uses YOLOv3 in conjunction with a Squeeze and Excitation Convolutional Neural Network (SENet-CNN); and Al Muskit et al [2], where researchers modified YOLOv3 with the addition of a Spatial Pyramid Pooling layer for robustness.

R-CNN Models, on the other hand, are **two stage** detectors. This means that one component of the model produces “candidate regions” which are then fed to another component which classifies the objects within these regions, discarding regions that do not contain sufficient evidence of the presence of an object [12]. Early R-CNN models proved to have high detection and classification accuracy, however, the models suffered from inefficiency due to the computational expense of producing and evaluating large numbers of candidate regions for a single image [41]. However, subsequent models, such as Fast R-CNN [11] and Faster R-CNN [29], have been produced that significantly improve on R-CNN’s efficiency [41].

Notable papers in the literature include Li et al [21] where researchers produced one of the early R-CNN model for fish identification; Zhao et al [39], where researchers modified an R-CNN model with the novel inclusion of an auxiliary backbone added to the standard R-CNN ResNet-based feature extractor; and Sun et al [34] where the researchers introduced an adaption of Faster R-CNN with a Random Vector Functional Link (RVFL), rather than Softmax, classifier to improve the model’s performance on small sample size training.

Table 1 displays the respective results of these models on both benchmark and custom datasets. The results in this table reveal that fish identification and classification from underwater video is a feasible task, with some detection and classification models achieving above 80% in at least one of accuracy, average precision, mean average precision and F-1 score on commonly accessible datasets. However, one can also observe from the table that it is currently difficult to compare competing fish-recognition models, with researchers training models on non-standard datasets and using a variety of different evaluation metrics.

Contrastive Learning for Fish Recognition. For completeness, a search of the available literature reveals only two previous works, M. Jahanbakht et al. [14] and Q. Yun et al. [37], that have directly applied contrastive learning techniques to fish recognition from underwater video. However, M. Jahanbakht et al. [14] use SimCLR-based contrastive learning as part of a larger pipeline and only focus on fish/no-fish detection; similarly, although Q. Yun et al. [37] focus on classification, they use a modified contrastive learning framework, with significant diversions from the SimCLR framework. As far as can be ascertained, no research has yet focused on applying the SimCLR framework to fish classification directly.

Paper	Task	Year	Dataset	Results Measured By	Results for Chosen Criteria
Jalal et al. [15]	Detection & classification	2020	LifeCLEF 2015 & custom dataset	F1-score	0.916 and 0.798 respectively
Knausgard et al. [18]	Detection & classification	2022	Fish4Knowledge & custom dataset	Accuracy	99.27% and 87.74% respectively
Xu et al. [36]	Detection only	2024	Custom dataset	Precision, Recall, mAP@50	97.19%, 98.42% and 0.982 respectively
Al Muskit et al. [2]	Detection only	2022	OzFish & DeepFish	F1-score	0.73 and 0.94 respectively
Li et al. [21]	Detection & classification	2015	LifeCLEF 2015 (subset)	mAP	0.814
Zhao et al. [39]	Detection & classification	2021	SeaCLEF 2017	AP	0.752
Sun et al. [34]	Detection only	2025	Custom dataset	mAP	0.898

Table 1: Comparison of Fish Identification Models

2.3 Contrastive Learning with SimCLR

Self Supervised Learning. ML research has begun to make strides towards effective learning from large unlabelled datasets, supplemented by small labelled ones [13]. These techniques work by training a DL model to recognise effective feature sets using a "pre-text task." These tasks use some form of contrived labels to train the model on, with the hope that the feature sets that the model learns in order to solve the contrived task can be usefully transferred to solving the primary task [13].

Contrastive Learning is a specific subset of SSL that formulates the pretext task as an exercise in trying to maximise the semantic similarity between related samples (called "positive pairs") while minimising the semantic similarity between unrelated samples (called "negative pairs") [4]. Generally, semantic similarity is measured by the absolute distance between two samples in the high dimensional feature space of a model - with high semantic similarity being a low euclidean or cosine distance and vice versa [4]. Different methods are used to extract positive and negative pairs from the unlabelled data, some approaches manufacture positive image pairs by extracting temporally related (close together in time) sample images from a video [31], SimCLR manufactures positive pairs by taking a single sample image and augmenting it heavily to form two separate but related "views" of the same image [8].

SimCLR. Along with its data augmentation approach, the SimCLR framework introduced two other novel contributions to contrastive learning. Firstly, the researchers showed that applying the contrastive loss function to the output of a non-linear head network attached to the final feature layer of the trainable network is more effective than applying the contrastive loss function directly to the final feature layer of the trainable network [8]. Additionally, the researchers found that the representational quality of the learned features improved when those features were non-linearly projected into a lower-dimensional feature space before the contrastive loss function was applied. Secondly, the researchers showed the effectiveness of the Normalized Temperature-scaled Cross Entropy Loss (NT-Xent loss) function for contrastive learning. These additions along with other, more minor, improvements allowed SimCLR to produce state of the art (SOTA) SSL performance results on benchmark image classification datasets such as ImageNet [13].

The structure of the SimCLR framework is illustrated in figure 1. For each image in a batch, that image is augmented to produce

the "positive pair" of related images. Next, these augmented images are passed through a ResNet classifier with the last layer removed, called an "encoder." This produces two 2048-dimensional feature vectors. These feature vectors are then passed to a non-linear Multi-Layer Perceptron (MLP) projection head to produce two 128-dimensional vector representations. The NT-Xent loss function is applied to these representations, updating the model to minimise the cosine distance between these two vectors, while maximising the cosine distance between one of these representation vectors and the representation vectors of all other samples in the batch (these being the "negative" samples).

The NT-Xent loss Function is a modified version of the classic Cross-Entropy loss function, with the addition of a temperature parameter and of terms to measure cosine similarity. Let $sim(y, z) = y^T z / (||y|| ||z||)$, i.e. the cosine similarity between vector y and vector z . Then, the NT-Xent loss, for a batch size of N , and for the

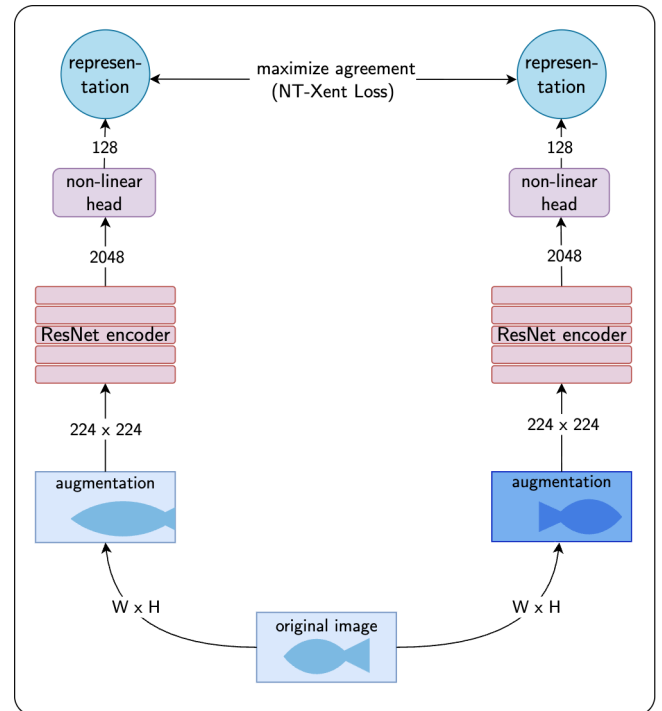


Figure 1: SimCLR Framework Structure

positive pair of samples x_j and x_k is defined as:

$$\ell_{x_j, x_k} = -\log \frac{\exp(\text{sim}(x_j, x_k)/\mathcal{T})}{\sum_{i=1}^{2N} \mathbb{1}_{[i \neq j]} \exp(\text{sim}(x_j, x_k)/\mathcal{T})}$$

Where $\mathbb{1}_{[i \neq j]}$ is an indicator function such that $\mathbb{1} = 1$ iff $i \neq j$ and 0 otherwise (effectively removing the case where one augmentation is being compared against itself). $\mathcal{T}$ denotes the *temperature* of the model which, put simply, scales how “confident” the model is in updating based on the outcome of its current predictions. Additionally, the summation in the numerator of the loss function sums over $2N$ samples, this is because *all* samples in the batch are augmented, producing double the number of negative examples.

3 METHOD

3.1 Data Preprocessing

SAIAB Dataset. The dataset provided to the team by the SAIAB posed a number of challenges for the effective application of DL. First and foremost, there was a severe lack of labelled data, with only $\sim 2,500$ partially labelled frames amounting to $\sim 5,500$ total labelled annotations. Secondly, these labels were in point annotation format, rather than the bounding box format required for training object detectors and classifiers. Thirdly, much of the footage was filmed in murky, heavily tinted water, significantly compromising visibility. Lastly, although multiple species are often present in a frame, each frame only contains annotations for a single species. It is this last fact that meant that we had to take a two stage detector-classifier approach. It would be impossible to train an all-in-one detector-classifier on our dataset as this would require images with all samples labelled. This meant that the team had to train a detector, and then a separate classifier for the detected “cutouts.”

The dataset contained three distinct sets of videos, filmed at three locations across a region of the East Coast of South Africa stretching from the city of East London to the town of Port Shepstone. The three datasets were labelled NSC-S (collected at the northernmost section of the region), WILD (collected at the southernmost section of the region), and Pondo (collected in the centre of the region). The dataset contained annotations corresponding to 156 distinct species of underwater animals (primarily fish species with the addition of some cephalopods and crustaceans).

Figure 2 shows a set of typical images from the dataset, displaying the high degree of inconsistency in colour and image quality across the dataset. Of the three sets of videos, the general visibility and quality of video was highest in the Pondo set. For this reason, we decided to focus experimentation only on this portion of the dataset. Unfortunately, the Pondo set was also the smallest and most sparsely annotated of the three. The entire Pondo set contains 17 $\sim$ hour long videos, with 430 annotated frames, for a total of 1,168 labelled point annotations. This is an exceptionally small dataset and a highly imbalanced one, with the largest class containing 265 point annotations and the smallest 23 classes containing 3 or fewer point annotations.

Preprocessing Labelled Samples. The first challenge the team encountered when preparing the ground truth dataset was the lack of bounding boxes. Therefore, the first step in data preprocessing was to convert the existing point annotations into labelled bounding



Figure 2: Sample Frames from SAIAB Dataset

boxes. Our initial approach was to use the open-source YOLO-Fish fish detection model [2] to predict bounding boxes, and then use a nearest neighbour type matching algorithm to match each point annotation to the bounding box with the centre closest to that point.

However, we found that YOLO-Fish performed poorly on our dataset, missing fish entirely, or predicting inaccurate bounding boxes. This, coupled with the nearest neighbour matching algorithm produced a small dataset with many mislabelled bounding boxes. Realising the importance of an accurate ground truth dataset, the team pursued a semi-automated approach to labelling the dataset. We used a tool that first proposed bounding boxes based on YOLO-Fish predictions, but then allowed team-members to manually modify, add, or delete bounding boxes. These bounding boxes were then attached the label of any point annotation within the boundaries of the bounding box. If a point annotation was within more than one bounding box, it was discarded; if a bounding box contained more than one point annotation, one of those point annotations was discarded. This was a relatively conservative matching algorithm, and meant that annotations were discarded quite easily. We decided, however, that this down-side was worth ensuring that all bounding boxes that we did use were assigned the correct labels. Figure 3 shows the bounding boxes produced for three typical images, with red bounding boxes remaining unlabelled, and green bounding boxes labelled using interior point annotations (each bounding box is also marked with the original confidence level predicted by the YOLO-Fish model).

This process allowed us to extract 5,088 labelled bounding boxes, 887 of which were from Pondo (with the remaining 281 point annotations discarded by the conservative matching algorithm) and 15,946 unlabelled bounding boxes. We used all 21,034 hand annotated bounding boxes to train a SOTA YOLOv11-nano object detector [17]. Using default hyper-parameters, this detector achieved a MaP@50 of 0.906 and a MaP@50-95 of 0.573.

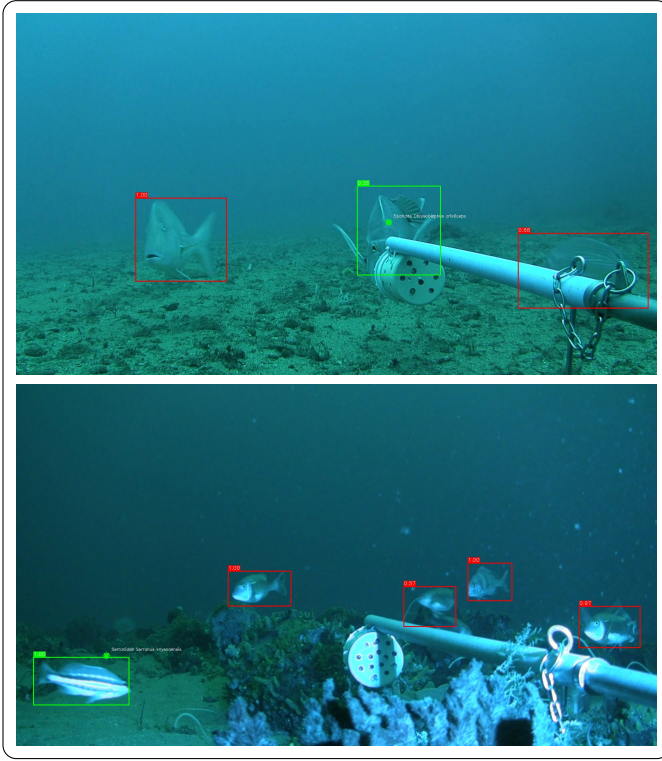


Figure 3: Sample Ground Truth Annotations

As we were only evaluating the SimCLR classifier using Pondo, and there was a relatively large species overlap between the three sets, we supplemented the 887 labelled bounding boxes from Pondo with bounding boxes from matching species in matching conditions (e.g. we did not supplement from murky videos) from the other two sets. This produced a dataset containing 1,926 labelled bounding boxes corresponding to 48 species classes. However, this dataset was unsuitable for training due to the fact that it still contained classes with fewer than three annotated samples, which is too few to both accurately train a classifier to recognise, and accurately evaluate this recognition. To solve this issue, all classes with under three samples were removed from the dataset. This filtered dataset was used for post-training the self-supervised classifier. Figure 4 displays the effect of the filtering process on the dataset. Different classes are represented by different colours, note that even after this filtering the dataset remains heavily imbalanced.

Processing Unlabelled Samples. For using the SimCLR framework to train a model, it was essential to build a large dataset of unlabelled images to train on. As the model would be classifying fish cutouts that had been pre-detected, it needed to be trained on an equivalent dataset. Therefore, to build this dataset, we used the YOLOv11 model that was trained on all hand-annotated bounding boxes to detect unlabelled fish cutouts from the underwater footage. From the entire Pondo footage, frames were sampled at 1 second intervals (to reduce the chance of very similar frames when sampling at shorter intervals). From these frames, the YOLOv11 model was used to predict bounding boxes. All bounding boxes with

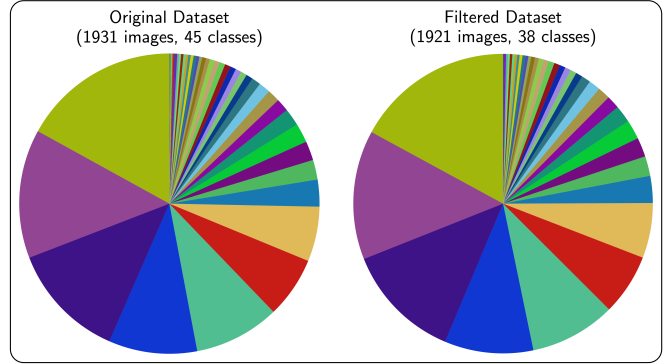


Figure 4: Classification Dataset Composition

confidence above 85% (a threshold which was found to produce almost no false positives) were extracted and stored as cutouts. This process produced 38,289 distinct cutouts. These unlabelled cutouts were used to train the model using the SimCLR framework. This process is displayed in figure 5.

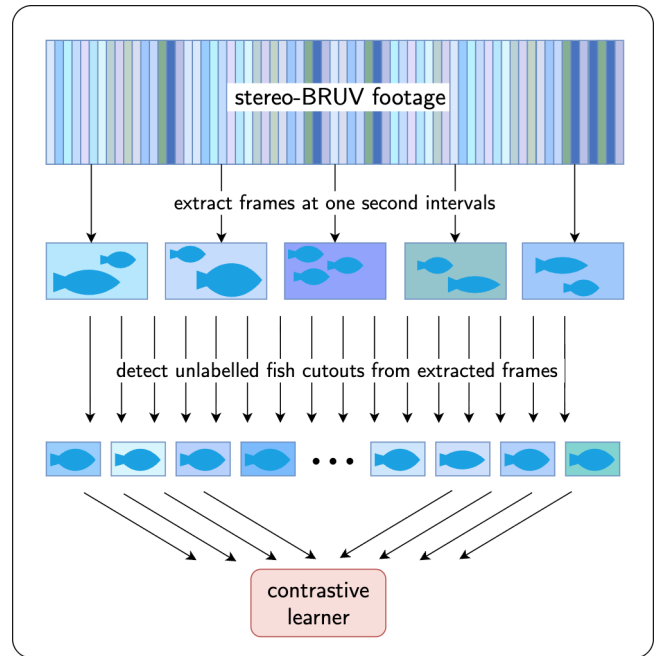


Figure 5: Unlabelled preprocessing Pipeline

3.2 SimCLR Implementation

The original SimCLR training framework was written using TensorFlow and optimised for running on Google Tensor Processing Units (TPU) [8]. The original code was written for Google v3 TPUs, whereas the team only had access to Google v4 TPUs. After many attempts to repurpose the original framework, we found that there were significant compatibility issues between the original code and the software underlying the newer v4 TPUs. Therefore, we decided

to rather use an open source PyTorch implementation of SimCLR that has been benchmarked against the original paper and shown to produce similar results [3].

Software Environment. The chosen implementation was produced in 2021, and thus depended on a variety of now deprecated versions of PyTorch and various other DL tools. Therefore, we created an Anaconda Virtual Environment [1] running Python 3.8, PyTorch 1.13.0 and TorchVision 0.14.0. All training and evaluation was run in this environment.

Hardware Environment. Computations were performed using facilities provided by the University of Cape Town’s ICTS High Performance Computing team: hpc.uct.ac.za – <https://doi.org/10.5281/zenodo.10021612>. All training and evaluation was performed on a single Nvidia L40 GPU.

Hyper-parameters. Each training run was performed with 50,000 steps (batches) with a batch size of 256 (the largest possible given the GPU memory). This is equivalent to ~ 33 epochs. This step-count was chosen as the largest training run that could viably be used in the experimentation cycle, given the time constraints, as each training run took ~ 20 hours on the GPU. No early stopping was employed, and as is shown in figure 6 below, performance on evaluation metrics tends to plateau at around 30,000 steps.

Hyper-parameter	Value	Description
Batch Size	256	Number of samples per training batch
Iterations	50,000	Number of training iterations
Learning Rate	1.0	Initial learning rate
Learning Rate Schedule	Warmup-Anneal	Learning rate scheduling strategy
Warmup Fraction	0.1	Fraction of training where warmup is applied
Weight Decay	1.0×10^{-6}	Regularisation coefficient
Optimiser	LARS	Layer-wise Adaptive Rate Scaling optimiser
Temperature	0.15	Temperature parameter for contrastive loss
Multiplier	2	Number of augmented images produced for each sample
Color Distortion Strength	1.0	Strength of color jittering augmentation
Scale Lower Bound	0.08	Lower bound for random crop scaling

Table 2: SimCLR Hyper-parameter Configuration

Given limited access to the HPC, a full hyper-parameter sweep was deemed too computationally expensive to be feasible. Therefore, initial experiments were done using the preset hyper-parameters used in the PyTorch framework to reproduce the original SimCLR results on the ImageNet dataset. However, further research revealed that for a batch size of 256, these were actually sub-optimal hyper-parameters. Bordes et al. [5] showed that higher learning rates and temperature values were capable of producing far superior results than were originally claimed in the SimCLR paper. Therefore, it was ultimately decided to use the hyper-parameters found by Bordes et al [5] to perform best on the ImageNet dataset with a batch size of 256. ImageNet was chosen as the images fell closest to the average resolution of the extracted unlabelled cutouts (ImageNet

uses a standard 224x224 pixel format). Table 2 displays the hyper-parameters chosen for the final evaluated model.

4 EVALUATION

4.1 Techniques

Linear Evaluation. The gold standard in evaluating SSL models is a technique called *linear evaluation* or *linear probing* [8]. This technique involves appending a single linear layer to the frozen backbone of the trained encoder [4]. This layer converts the output dimension of the encoder (2048 in this case) to a vector with cardinality equal to the number of classes being classified (38 in this case). This linear layer is trained on the small labelled dataset using one-hot encodings of the class labels of each sample.

This evaluation measures the expressive power of the features that the model has learned through SSL training. If a model has learned to produce effective features, then it is assumed that these features can be used to effectively linearly separate classes. This method may not produce the optimal classifier, but it gives experimenters a very good sense of the representational quality of the feature vector produced by the encoder [4].

Non-linear Evaluation. Non-linear evaluation involves appending a non-linear classification head to the output of the frozen backbone and training these layers on the labelled dataset [4]. This technique is less indicative of the quality of the SSL training run, but can produce higher accuracy than pure linear evaluation [4]. For our non-linear evaluation, we appended a single hidden linear layer of dimension 2048 to the frozen backbone of the encoder, followed by a non-linear layer that used the ReLU activation function to convert the output of the hidden layer to a vector with cardinality equal to the number of classes being classified (38 in this case).

Fine-tuning. We also tested the performance of the encoder when allowing for the ResNet parameters to be partially or fully fine-tuned on the labelled dataset. We appended a single linear layer to the output of the encoder, transforming this into a vector of dimension 38, and allowed for either the parameters in the last two convolution blocks (partial fine-tuning), or the entire encoder (full fine-tuning) to be updated during training on the labelled dataset.

4.2 Experimental Setup

We used linear and non-linear protocols and partial and full fine-tuning when evaluating the final models trained using the SimCLR framework. The labelled dataset was split randomly into three subsets: train (70%), validate (15%), and test (15%). The validation data was used for hyper-parameter tuning and model selection, with the test set reserved for final performance evaluation. In addition to evaluating the models trained via SimCLR on the Pondo dataset, three comparative benchmarks were established.

First, we evaluated a ResNet-50 model pre-trained on the ImageNet dataset via supervised learning. We performed the linear evaluation protocol on this model using the Pondo dataset. This test was designed to assess the quality of the feature sets learnt through SimCLR training. If the SimCLR models performed better than the baseline ResNet, this would suggest that the SimCLR training produced models with feature sets that were more useful to our

specific classification task than the general feature sets learnt by an ImageNet-trained ResNet model.

Secondly, we trained and evaluated a YOLOv11-n classifier model, a single object classifier from the suite of YOLOv11 models [17], on the labelled Pondo dataset. This allowed us to compare our approach to the current SOTA in supervised learning.

Finally, we trained and evaluated a model on the benchmark Fish4Knowledge dataset [9] using the SimCLR framework. The original Fish4Knowledge dataset is made up of 27,370 images of labelled fish cutouts, each classified into 1 of 20 classes. We discarded the labels of 90% of the dataset and used these unlabelled images for SimCLR training. We then performed linear and non-linear evaluation using the remaining labelled 10% of the dataset. Our dataset is geographically specific, and this test was designed to evaluate whether our method generalised to diverse underwater conditions.

During early experimentation, we noticed that accuracy for ResNet evaluation and YOLO performance varied significantly across different random seeds (up to a difference of 2%). In fact, this is a known phenomenon [26] when training DL models on very small labelled datasets. Therefore, for statistical robustness, in all experiments other than the Fish4Knowledge evaluation, ten repeated evaluations were performed, each with a different random seed and with the same set of random seeds being used across all evaluations. Along with reducing the dependence of our results on the arbitrary choice of a random seed, this repetition allowed us to perform a simple sign test to robustly compare the classification performance of different models.

Tables 3 and 4 display the hyper-parameters used for linear and non-linear evaluation of all models respectively. For the sake of fairness and simplicity, we decided to tune hyper-parameters only once and apply these parameters across all evaluations.

Hyperparameter	Value
Batch Size	16
Test Batch Size	128
Epochs	80
Learning Rate	0.001
Weight Decay	0.0001
Optimiser	Adam

Table 3: Linear Eval Hyper-parameters

Parameter	Value
Batch Size	16
Test Batch Size	128
Epochs	80
Learning Rate	0.001
Weight Decay	0.0001
Optimiser	Adam

Table 4: Non-linear Eval & Fine-tune Hyper-parameters

5 RESULTS & DISCUSSION

5.1 Evaluation of SimCLR Training

The model successfully learned effective representations through contrastive training, as demonstrated in Figure 6, which presents linear and non-linear evaluation and partial and full fine-tuning results at 5,000-step intervals (with error bars representing standard deviation across the ten seeds). Accuracy across all evaluations improves monotonically until step 25,000, indicating strong initial learning. Subsequently, accuracy for all metrics oscillates around the same values, with no metric deviating by more than a standard deviation in accuracy from steps 25,000 to 50,000. This plateau suggests that while the model acquires effective representations

early in SimCLR training, continued training beyond approximately 25,000 steps does not improve generalisation and may indicate overfitting.

Of the four evaluation types performed, non-linear evaluation clearly produces the lowest accuracy. This is likely due to the parameters in the non-linear head over-fitting to the training set. This over-fitting does not affect the linear evaluation to such a large extent due to the limited capacity for a single linear layer to “memorise” the training set. Both partial and full fine-tuning outperform linear evaluation slightly, with full fine-tuning performing best of all at step 25,000, with a mean accuracy of 70.65%. This is due to the fact that fine-tuning allows the representation spaces of the models to be modified to become more suited to the specific task at hand (classification), rather than the “pretext task” which, because of its loss function, likely benefits from a slightly different representation space. This phenomenon was expected as fine-tuning was shown to outperform linear evaluation in the original SimCLR paper [8].

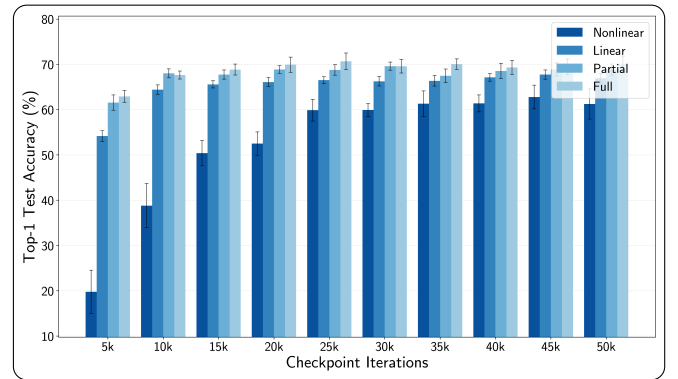


Figure 6: Evaluation of SimCLR Models on Pondo Dataset

5.2 Comparison to ImageNet ResNet

As can be seen in figure 7, the features learnt by a SimCLR pre-trained model are better suited to the task than those learnt by the supervised ImageNet pre-training. Linear evaluation of the SimCLR pre-trained encoder produces an average accuracy > 15% higher than linear evaluation of the ImageNet encoder. This difference is far greater than a standard deviation in performance from either model (indicated by the error bars). This suggests the general efficacy of the SimCLR framework for self-supervised pre-training.

5.3 Comparison to Fully Supervised YOLOv11

As evidence by figure 9, the SOTA YOLOv11-n model trained via supervised training on the Pondo dataset performs with very similar top-1 accuracy to a SimCLR pre-trained fine-tuned ResNet model across the ten random seeds. To test whether there is a statistically significant difference between the performance of these two models, we used a simple sign test. Using the null hypothesis H_0 that these two models perform equally, it would be expected that, for a single seed, the difference $X = Accuracy_{YOLO} - Accuracy_{SimCLR}$ would be equally likely to have a positive or negative sign. Therefore,

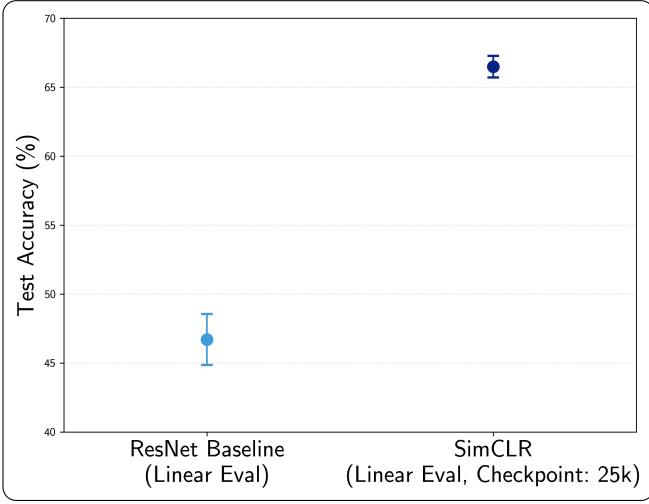


Figure 7: Linear-evaluation Comparison with ImageNet Pre-trained ResNet

under the null hypothesis, the number of seeds where YOLO outperforms SimCLR would follow the binomial distribution $B(10, 0.5)$. With 4 positive differences (YOLO > SimCLR) out of 10 seeds, the two-tailed p-value was 0.75, far greater than our significance level $\alpha = 0.05$, indicating no statistically significant difference between the classification accuracy of the two models.

These two models employ highly different architectures and training regimes. The high similarity in classification accuracy between these two models suggests that there may simply be a practical upper limit on the possible accuracy attainable by any vision model on the Pondo dataset. With this accuracy achievable in 2025 through SOTA few-shot learners, and only possible through longer, semi-supervised training for older architectures. Interestingly, as can be seen in figure 8, the confusion matrices of the best performing YOLO and SimCLR-trained ResNet models are similar, but not exactly the same. This shows that the models, although statistically indistinguishable in accuracy, learnt different decision boundaries.

Model	Accuracy	Precision	Recall	F1-Score
SimCLR	0.7388	0.6943	0.7388	0.7101
YOLOv11-n	0.7285	0.7155	0.7285	0.7162

Table 5: Performance Metrics: SimCLR & YOLO

For completeness, table 5 presents detailed metrics of the best fine-tuned SimCLR model and the best YOLOv11 model across the ten seeds. However, there is little variation both between models and across metrics. This precludes us from making additional strong claims about performance differences between the models.

5.4 Evaluation on Fish4Knowledge Dataset

As is evidenced by figure 10, the self-supervised models performed well on the Fish4Knowledge dataset. Attaining > 96% linear evaluation and fine-tuning top-1 accuracy after only 5,000 steps and

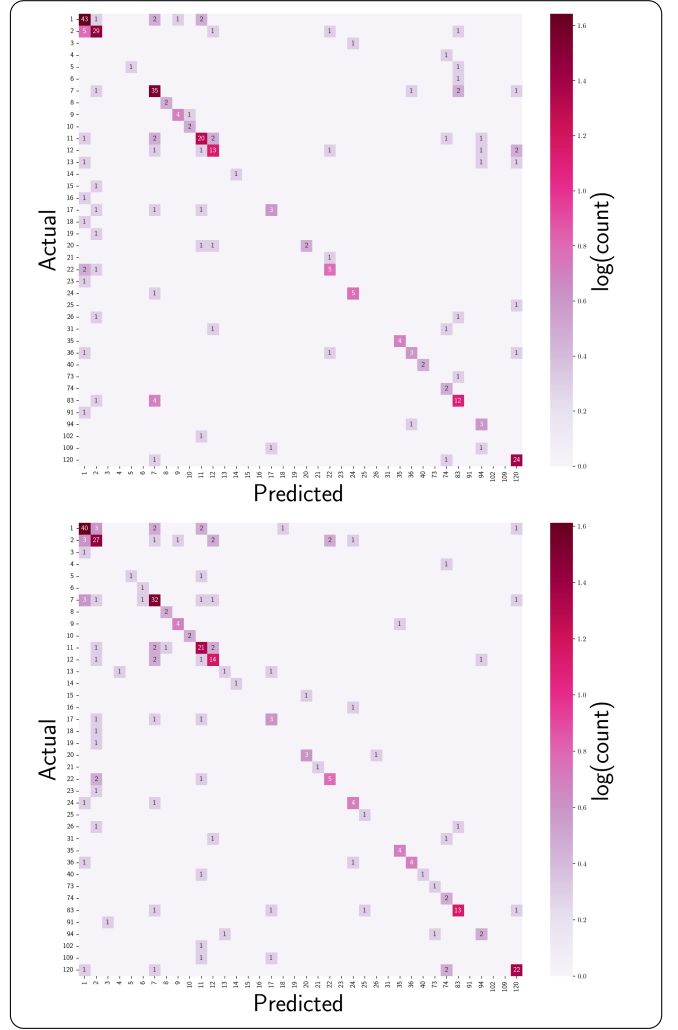


Figure 8: Confusion Matrices of Best Performing Models: SimCLR-ResNet (top) and YOLOv11 Classifier (bottom)

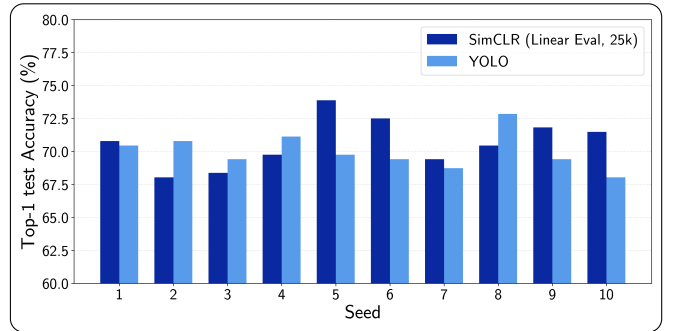


Figure 9: Comparison with YOLOv11-n Classifier

reaching > 98% top-1 accuracy on both metrics at subsequent checkpoints, even with access to only 10% of the dataset’s labelled

annotations. This shows that the framework generalises well, and points to the difficulty of the SAIAB dataset in comparison with benchmark datasets.

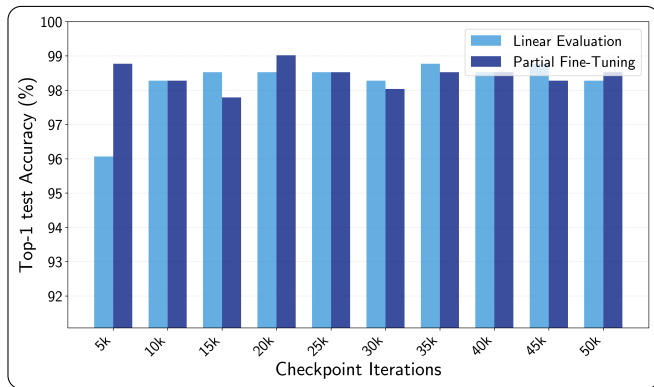


Figure 10: Evaluation of SimCLR Models on Fish4Knowledge Dataset [9]

5.5 Limitations

There were a variety of shortcomings in our experimental method, some unsurpassable and some surpassable only with the benefit of hindsight. Most notably, due to the format of our dataset, we were limited in the types of evaluation we could perform. The intention of the project was to investigate detection and classification tools for the SAIAB dataset. However, because the team only had access to partially annotated frames, we did not have access to a ground truth dataset to test detection and classification against. Any attempt to use the labelled data would produce results overpopulated with false-negatives wherever the model being tested detects a fish of any species other than the single species annotated in a frame. This meant that we were only able to evaluate detection and classification as two separate tasks. Therefore, we are only able to provide an incomplete account of the efficacy of our approach to the fundamental task at hand.

Further limitations included, first, those due to resource constraints, notably, (i) the inability to perform a complete hyper-parameter sweep for the SimCLR training, and (ii) the inability to perform SimCLR training with larger batch-sizes, which would have required consistent access to multiple GPUs. And, second, those due to experimental decisions, notably, the fact that although the final dataset chosen to evaluate the model on is likely more representative of the SAIAB data than most (if not all) publicly available data, due to layers of filtering (choosing to evaluate only the Pondo subset, and only the subset of classes from Pondo with sufficient samples) it is less representative of the SAIAB data than could be possible with less aggressive filtering.

5.6 Additional Insights

The most interesting incidental result that emerged from experimentation was that the best SimCLR trained ResNet model performed with a statistically insignificant difference in top-1 classification accuracy to the purely supervised YOLOv11-n classifier model. This

result may suggest a hard ceiling on the best possible classification accuracy attainable on the filtered Pondo dataset. This is not an infeasible conclusion given the extreme difficulty of the dataset at hand. A number of the fish cutouts in the test-set may simply be impossible to classify based on the available training data, and may require more contextual clues.

Additionally, it was interesting to note that the performance of the SimCLR trained model peaked far earlier in training than was expected. The performance of the model on both linear evaluation and fine-tuning peaked significantly before the end of the ~ 33 epochs of training. This is unexpected given that in the original SimCLR paper, there was a clear positive correlation between number of training epochs and classification accuracy lasting well into hundreds of epochs of training [8]. This early plateau may be due to the fact that we trained the model with a far smaller unlabelled dataset than the one used in the original SimCLR paper.

5.7 Future Research

The clearest direction for future research is simply the expansion of the dataset that is used for training and testing to encompass all available data. This would involve extracting additional unlabelled data from the WILD and NSC-S dataset for unsupervised training and using all available labelled data for evaluation.

Additionally, it would be valuable to test a variety of models, with a variety of hyper-parameters, on the Pondo dataset to assess whether the postulated upper bound on classification accuracy truly exists.

Lastly, given that we have shown that SSL is suitable for the task of fish classification, it may be valuable to investigate more current SSL frameworks such as the recently released DINOv3 [32] model. A model which authors claim to surpass the previous SOTA in self-supervised learning and come close to matching the SOTA in supervised learning [32].

6 CONCLUSION

In this work, we have shown the potential for the application of the SimCLR contrastive learning framework to the task of fish classification from underwater video. Specifically we have shown that by using the SimCLR framework we were able to train a ResNet model that achieved a mean classification accuracy (over ten random seeds) of 70.65% on a subset of the novel SAIAB dataset of underwater footage.

This basic result suggests that there is potential for further work in SSL for fish recognition from underwater video. A task which, due to a general paucity of labelled datasets, may at times be unsuitable for classic supervised DL solutions.

However, we have also shown that modern supervised models can very effectively learn to classify underwater objects, even when trained on very small labelled datasets. With the fully supervised YOLOv11-n matching the classification accuracy of our SimCLR-trained ResNet while requiring far less training resources. This suggests that both investigation into modern SSL techniques and additional data collection to enhance supervised learning would be reasonable avenues for the SAIAB to pursue further.

REFERENCES

- [1] Anaconda software distribution, 2020.
- [2] Abdullah Al Muksit, Fakhru Hasan, Md Fahad Hasan Bhuiyan Emon, Md Rakibul Haque, Arif Reza Anwary, and Swakkhar Shatabda. Yolo-fish: A robust fish detection model to detect fish in realistic underwater environment. *Ecological Informatics*, 72:101847, 2022.
- [3] Andrew Atanov. Simclr pytorch implementation. <https://github.com/AndrewAtanov/simclr-pytorch>, 2020.
- [4] Randall Balestrieri, Mark Ibrahim, Vlad Sobal, Ari Morcos, Shashank Shekhar, Tom Goldstein, Florian Bordes, Adrien Bardes, Gregoire Mialon, Yuandong Tian, et al. A cookbook of self-supervised learning. *arXiv preprint arXiv:2304.12210*, 2023.
- [5] Florian Bordes, Randall Balestrieri, and Pascal Vincent. Towards democratizing joint-embedding self-supervised learning. *arXiv preprint arXiv:2303.01986*, 2023.
- [6] Ma a Cappel, Euan Harvey, H Malcolm, P Speare, et al. Potential of video techniques to monitor diversity, abundance and size of fish in studies of marine protected areas. *Aquatic Protected Areas-what works best and how do we know*, 1:455–464, 2003.
- [7] Mike Cappel, E Harvey, and M Shortis. Counting and measuring fish with baited video techniques-an overview. In *Australian Society for Fish Biology Workshop Proceedings*, volume 1, pages 101–114. Australian Society for fish biology Tasmania, 2006.
- [8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PMLR, 2020.
- [9] Robert B Fisher, Yun-Heh Chen-Burger, Daniela Giordano, Lynda Hardman, Fang-Pang Lin, et al. *Fish4Knowledge: collecting and analyzing massive coral reef fish video data*, volume 104. Springer, 2016.
- [10] Chenping Fu, Risheng Liu, Xin Fan, Puyang Chen, Hao Fu, Wanqi Yuan, Ming Zhu, and Zhongxuan Luo. Rethinking general underwater object detection: Datasets, challenges, and solutions. *Neurocomputing*, 517:243–256, 2023.
- [11] Ross Girshick. Fast r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 1440–1448, 2015.
- [12] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 580–587, 2014.
- [13] Jie Gui, Tuo Chen, Jing Zhang, Qiong Cao, Zhenan Sun, Hao Luo, and Dacheng Tao. A survey on self-supervised learning: Algorithms, applications, and future trends. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):9052–9071, 2024.
- [14] Mohammad Jahanbakhsh, Mostafa Rahimi Azghadi, and Nathan J Waltham. Semi-supervised and weakly-supervised deep neural networks and dataset for fish detection in turbid underwater videos. *Ecological Informatics*, 78:102303, 2023.
- [15] Ahsan Jalal, Ahmad Salman, Ajmal Mian, Mark Shortis, and Faisal Shafait. Fish detection and species classification in underwater environments using deep learning with temporal information. *Ecological Informatics*, 57:101088, 2020.
- [16] Muwei Jian, Nan Yang, Chen Tao, Huixiang Zhi, and Hanjiang Luo. Underwater object detection and datasets: a survey. *Intelligent Marine Technology and Systems*, 2(1):9, 2024.
- [17] Rahima Khanam and Muhammad Hussain. Yolov11: An overview of the key architectural enhancements. *arXiv preprint arXiv:2410.17725*, 2024.
- [18] Kristian Muri Knausgård, Arne Wiklund, Tonje Knutsen Sordalen, Kim Tallaksen Halvorsen, Alf Ring Kleiven, Lei Jiao, and Morten Goodwin. Temperate fish detection and classification: a deep learning based approach. *Applied Intelligence*, 52(6):6988–7001, 2022.
- [19] Timothy J Langlois, Benjamin R Fitzpatrick, David V Fairclough, Corey B Wakefield, S Alex Hesp, Dianne L McLean, Euan S Harvey, and Jessica J Meeuwig. Similarities between line fishing and baited stereo-video estimations of length-frequency: novel application of kernel density estimates. *PLoS One*, 7(11):e45973, 2012.
- [20] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.
- [21] Xiu Li, Min Shang, Hongwei Qin, and Liansheng Chen. Fast accurate fish detection and recognition of underwater images with fast r-cnn. In *OCEANS 2015-MTS/IEEE Washington*, pages 1–5. IEEE, 2015.
- [22] Marine Protected Areas South Africa. Protecting the ocean. <https://www.marineprotectedareas.org.za/protecting-the-ocean>, 2024. Accessed: 2025-08-18.
- [23] Daniel Marrable, Kathryn Barker, Sawitchaya Tippaya, Mathew Wyatt, Scott Bainbridge, Marcus Stowar, and Jason Larke. Accelerating species recognition and labelling of fish from underwater video with machine-assisted deep learning. *Frontiers in Marine Science*, 9:944582, 2022.
- [24] David Maxwell and Simon Jennings. Power of monitoring programmes to detect decline and recovery of rare and vulnerable fish. *Journal of Applied Ecology*, 42(1):25–37, 2005.
- [25] Papers With Code. Coco test-dev benchmark (object detection) on papers with code. <https://paperswithcode.com/sota/object-detection-on-coco>, 2025. Accessed: March 21, 2025.
- [26] David Picard. Torch. manual_seed (3407) is all you need: On the influence of random seeds in deep learning architectures for computer vision. *arXiv preprint arXiv:2109.08203*, 2021.
- [27] Xiangrong Qin, Changdong Yu, Baisheng Liu, and Zhihao Zhang. Yolo8-fasg: A high-accuracy fish identification method for underwater robotic system. *Ieee Access*, 2024.
- [28] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- [29] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, 28, 2015.
- [30] Alzayat Saleh, Marcus Sheaves, and Mostafa Rahimi Azghadi. Computer vision and deep learning for fish classification in underwater habitats: A survey. *Fish and Fisheries*, 23(4):977–999, 2022.
- [31] Pierre Sermanet, Corey Lynch, Yevgen Chebotar, Jasmine Hsu, Eric Jang, Stefan Schaal, Sergey Levine, and Google Brain. Time-contrastive networks: Self-supervised learning from video. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 1134–1141. IEEE, 2018.
- [32] Oriane Siméoni, Huy V Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Quab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025.
- [33] South African Institute for Aquatic Biodiversity. South african institute for aquatic biodiversity (saiab). <https://saiab.ac.za/>, 2025. Accessed: 2025-04-15.
- [34] He Sun, Aming Yue, Wenhua Wu, and Haitian Yang. Enhanced marine fish small sample image recognition with rvfl in faster r-cnn model. *Aquaculture*, 595:741516, 2025.
- [35] Shubo Xu, Minghua Zhang, Wei Song, Haibin Mei, Qi He, and Antonio Liotta. A systematic review and analysis of deep learning-based underwater object detection. *Neurocomputing*, 527:204–232, 2023.
- [36] Xiaobin Xu, Jinchao Hu, Jian Yang, Yingying Ran, and Zhiying Tan. A fish detection and tracking method based on improved inter-frame difference and yolo-cts. *IEEE Transactions on Instrumentation and Measurement*, 2024.
- [37] Qiankun Yan and Xiujuan Du. Classification of underwater fish image based on supervised contrastive learning. In *2024 International Symposium on Intelligent Robotics and Systems (IROS)*, pages 48–53. IEEE, 2024.
- [38] Jian Zhang, Ruiteng Zhang, Xinyue Yan, Xiting Zhuang, and Ruicheng Cao. Bg-yolo: A bidirectional-guided method for underwater object detection. *arXiv preprint arXiv:2404.08979*, 2024.
- [39] Zhenxi Zhao, Yang Liu, Xudong Sun, Jintao Liu, Xinting Yang, and Chao Zhou. Composited fishnet: Fish detection and species recognition from low-quality underwater videos. *IEEE Transactions on Image Processing*, 30:4719–4734, 2021.
- [40] Tao Zheng, Junfeng Wu, Han Kong, Haiyan Zhao, Boyu Qu, Liang Liu, Hong Yu, and Chunyu Zhou. A video object segmentation-based fish individual recognition method for underwater complex environments. *Ecological Informatics*, 82:102689, 2024.
- [41] Zhengxia Zou, Keyan Chen, Zhenwei Shi, Yuhong Guo, and Jieping Ye. Object detection in 20 years: A survey. *Proceedings of the IEEE*, 111(3):257–276, 2023.